Tiger Workshop

Einführung

Der Workshop soll einen kleinen Blick in Tiger hinein ermöglichen. Tiger ist ein Testframework. Der Fokus und die Stärke ist das **Interface-Blackbox-Testing** von Komponenten. Aufgrund seiner flexiblen Anlage ermöglicht Tiger damit eine Vielzahl an Testszenarien und Einsatzmöglichkeiten.

Um diese Einführung so praxisnah und einfach wie möglich zu gestalten wird hier beispielhaft eine kleine Testsuite für das Abtesten einer externen Webseite gebaut. In vielen kleine Schritten, sogenannten **Stages**, werden verschiedene Techniken von Tiger vorgestellt. Der Teilnehmer soll, allein oder in einer Gruppe, zunehmend kompliziertere Techniken einsetzen, um das System abzutesten. Wer einmal nicht mitkommt, kann für die nächste Stage einfach den entsprechenden Branch auschecken und weitermachen. Abgerundet wird der Aufbau durch **Zusatzaufgaben** (für die ganz Fleißigen).

Zur Einführung gibt es noch eine **Pre-Stage**, welche einen Einstieg in das Test-Projekt bietet. Der Fokus hier ist mehr maven & serenity und weniger Tiger. Der eigentliche Workshop beginnt daher NACH der Pre-Stage.

Pre-Stage: Hello World

Hier gibt es zwei Möglichkeiten die Pre-Stage aufzusetzen. Entweder ihr holt euch das Repo (https://github.com/gematik/tiger-workshop) mit dem Branch `workshop/stage1` oder ihr setzt ein neues Projekt auf. Wenn ihr ein neues Projekt aufsetzen wollt, setzt die folgenden Schritte um:

- Lege ein neues Projekt an, z.B. "tiger-workshop".
- Füge in der Pom als parent pom die tiger-starter-parent mit der neuesten Tigerversion ein (somit ist sichergestellt das auch die entsprechenden Plugins zur Verfügung stehen und man braucht sich nicht um die Dependency-Updates zu kümmern, das macht das TigerTeam):

```
  <parent>
       <groupId>de.gematik.test</groupId>
       <artifactId>tiger-starter-parent</artifactId>
       <version>4.3.1</version>
   </parent>
```

- Lege unter `src/test/resources/features` eine neue `helloWorld.feature` Datei an
- In der Datei soll folgendes eingetragen werden:

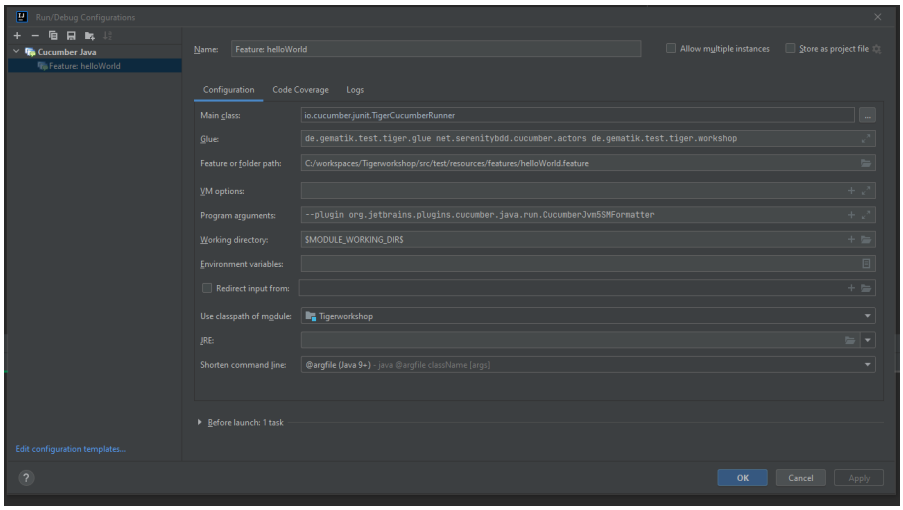
```
Feature: Hello World
  Scenario: is everything up and running
    Given TGR show banner "Hello World!"
```

- Im Rootverzeichnis erstellt ihr eine vorerst leere `tiger.yaml` Datei.
- Das Ganze soll mit "`mvn verify`" abgeprüft werden.

Hinweis für später im eigen Setup: Sollte man in seinem Projekt das tiger-Projekt nicht als parent aufsetzen können, dann nimmt man statt dessen die tiger-bom in die dependencyManagement auf, wie im User Manual im Kapitel Without the Tiger Start Pom beschrieben.

Ausführen in der IDE (hier: IntelliJ, prinziell kann jede IDE verwendet werden (VSCode hat allerdings keine guten Plugins für Cucumber, weshalb wir IntelliJ empfehlen, oder ihr führt Tiger in VSCode über das Terminal aus)

- Eine neue Run-Configuration für Cucumber-Java anlegen (alternativ kann das Template unter benutzt werden: https://github.com/gematik/tiger-workshop/blob/workshop/stage1/.idea/runConfigurations/_template__of_Cucumber_Java.xml)
  - Main Class auf *io.cucumber.junit.TigerCucumberRunner* abändern
  - als Glue folgende Pakete eintragen
    - de.gematik.test.tiger.glue
    - net.serenitybdd.cucumber.actors
    - de.gematik.test.tiger.workshop
  - Program arguments eintragen
    - --plugin org.jetbrains.plugins.cucumber.java.run.CucumberJvm5SMFormatter
  - Für Windows-User wichtig: die Kommandozeile verkürzen @argfile (Java 9+)



Hinweis: Geht man auf "Edit configuration templates..." unten links im Bild, kann man dort die Defaultwerte für MainClass, GlueCode, etc eintragen. So muss man die Run Configuration nicht jedes Mal neu aufsetzen, wenn man z.B. ein neues Feature File laufen lassen will.

Für IntelliJ ist es sinnvoll das "Cucumber+" und das "Cucumber for Java" Plugin zu installieren, ggf auch noch "Gerkin".

Serenity BDD Exkurs

Serenity BDD besteht in der Regel aus sogenannten .feature Files, welche im Gherkin-Format geschrieben sind, und dazu dienen, Anforderungen in verständlicher Sprache zu beschreiben. Das Feature ist das zu testende Hauptthema. Ein Scenario beschreibt einen konkreten Anwendungsfall. Die Schlüsselwörter "Angenommen/Und/Wenn/Dann (Given/And/When/Then auf Englisch)" strukturieren die Schritte eines Scenarios.

Der Clou dabei ist, dass diese Tests auch von Managern und Nicht-Softwareentwickler verstanden werden, da sie in fast natürlicher Sprache geschrieben werden. Und zwar derart:

.feature File

```
Feature: Benutzer-Login

  Als registrierter Benutzer
  möchte ich mich auf der Webseite einloggen können
  um Zugriff auf meine persönlichen Daten zu erhalten.

  Szenario: Erfolgreicher Login mit gültigen Zugangsdaten
    Angenommen der Benutzer befindet sich auf der Login-Seite
    Und der Benutzer gibt die E-Mail "benutzer@example.com" ein
    Und der Benutzer gibt das Passwort "geheimespasswort" ein
    Wenn der Benutzer auf "Login" klickt
    Dann sollte der Benutzer auf die Startseite weitergeleitet werden
    Und eine Willkommensnachricht "Willkommen, Benutzer!" sollte angezeigt werden
```

Der sogenannte GlueCode enthält die eigentliche Implementierung der Tests von z.B. "der Benutzer befindet sich auf der Login-Seite". Es sind einer oder mehreren Javaklassen, die sich im `test` Package befinden.

GlueCode

```
@Angenommen("der Benutzer befindet sich auf der Login-Seite")
public void benutzer_auf_login_seite() {
    // Hier wird im Test die Login-Seite geöffnet
}
```

Stage 1: Healthcheck ans System

Unter https://github.com/gematik/tiger-workshop findet ihr den Tiger-Workshop, wo ihr euch das Projekt auschecken könnt. Es gibt für jeden Stage einen Branch, so dass ihr jederzeit diesen auschecken könnt und weitermachen könnt, auch wenn ihr die letzte Stage nicht mitgemacht habt.

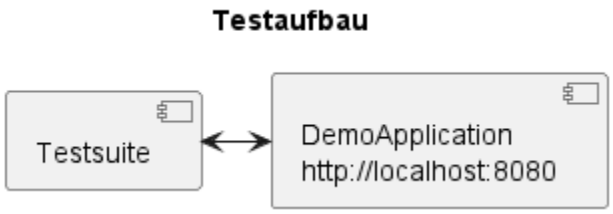
Für den Tigerworkshop haben wir folgenden Testaufbau: Unser Testprojekt kommuniziert mit einem einfachen Webserver und stellt Anfragen an das System. Der Webserver ist für uns eine Blackbox, dessen Restschnittstellen wir benutzen wollen. Der Tiger startet zuvor die DemoApplication als external Jar.

Erstellt in der `tiger.yaml` einen neuen Server mit Namen "`demo`" vom Typ "`externalJar`". Das demo jar liegt im `main` Branch des Projekt tiger-workshop. Gebt die Source in der `tiger.yaml` entsprechend mit local: Path-zum-Jar an. Die healthcheck-URL ist `http://localhost:8080/service/status` (Bei Schwierigkeiten kannst du im User Manual Abschnitt externalJar Hilfe bekommen).

Die DemoApplication wird dann unter http://localhost:8080/ erreichbar sein, die OpenApi unter http://localhost:8080/swagger-ui/index.html.

Jetzt wollen wir eine REST Anfrage an diese URL schicken und die Antworten auswerten.

Dafür erstellen wir einen Gluecode Step, wo wir z.B. über SerenityRest die URL (z.B http://localhost:8080/service/status) abfragen. Daran denken, dass das Verzeichnis mit in den GlueCode bei der Run Configuration mit aufgenommen werden muss.



Stage 2: Tiger Proxy

Bislang haben wir den Server direkt angesprochen. Nun werden wir den Tiger-Proxy nutzen. Der Tiger-Proxy ist ein Proxy-Server, welcher den Verkehr einfach weiterleitet und für uns später in der Testsuite zur Verfügung stellt. Dabei kann er mehrere Quellen vereinigen, vielfältige Strukturen parsen und Daten strukturiert zur Verfügung stellen. Am Schluss kann er für uns eine übersichtliche Datei bereitstellen, in welcher der gesamte Netzwerkverkehr aufbereitet nachlesbar ist.

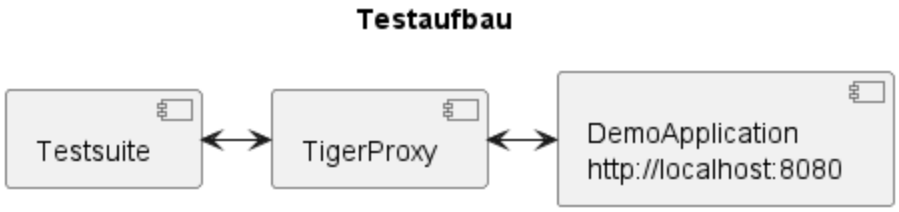
In einer normalen Tiger-Testumgebung wird immer ein Tiger-Proxy gestartet, der sogenannte lokale Tiger-Proxy. Der Traffic, der über den lokalen Tiger-Proxy läuft, steht für uns direkt in der Testsuite zur Verfügung. Weiterhin wird er beim Starten als Default-Proxy gesetzt, wodurch er schon in der vorigen Stage die gesamte Kommunikation mitgeschnitten und weitergeleitet hat. Beim Start wird außerdem für jeden Server, der Teil der Testumgebung ist, eine Route gesetzt. Domain-Name ist hierbei der Servername aus der `tiger.yaml`.

Wir wollen nun den Tiger-Proxy explizit verwenden, den aufgezeichneten Traffic anschauen und auf diesem direkt Assertions durchführen.

- Dazu ändern wir zunächst den REST-Aufruf im Gluecode: Von `http://localhost:8080` zu http://demo/
    - *Dadurch abstrahieren wir von dem konkreten Server. Die Testsuite weiß nicht mehr, welcher Demoserver selbst angesprochen wird. Das macht ein nachträgliches Austauschen einfacher.*
- Neuen Schritt in der Testsuite einfügen `TGR find first request to path ""` (Den Pfad entsprechend ergänzen. **Wichtig:** Hier ist nur der Pfad einzutragen, nicht die gesamte URL. Hostname und Protokoll können also entfallen)
    - *Dieser Schritt verschiebt den Zeiger des aktuellen Nachrichtenpaars, welches in der Testsuite betrachtet wird. "current request" und "current response" nutzen immer diesen Zeiger. Neue empfangene Nachrichten verschieben den Zeiger NICHT automatisch! Mehr dazu in der nächsten Stage.*
- ```
  TGR current response body matches:
  """
  OK
  """
  ```
    - *Dies vergleicht den HTTP-Body der current Response mit dem gegebenem String. Dies kann ein exakter Match oder ein Regex sein.*

Tipp: Tiger hat viele vordefinierte Gluecode Anweisungen, die auch ständig erweitert werden. Bitte schaut euch bei Interesse im Tigerprojekt die vorhandenen Klassen an.

Debug-Tipp: Im Verzeichnis `target/rbellogs` stehen nun die Aufzeichnungen des Verkehrs bereit, der über den Tiger-Proxy geleitet wurde. Dieser ist sauber nach Szenarien getrennt und eine gute Hilfe, wenn mal etwas nicht auf Anhieb funktioniert (Sehen wir dort überhaupt den Verkehr? Wie sehen die Nachrichten aus? Was ist die Fehlermeldung des Servers?)
  ```



Stage 3: Current request/response

Intern merkt sich die Testsuite alle Nachrichten, die der Tiger-Proxy auffängt. In diesen Nachrichten gibt es wiederum einen Zeiger, der "current request /response" Zeiger. Er kann nur durch Suchfunktionen bewegt werden (bewegt sich also NICHT durch das Auffangen einer neuen Nachricht automatisch weiter). Um gut mit den Testschritten der Art "current request matches" arbeiten zu können müssen wir diesen Zeiger korrekt bewegen. Das können wir mit den Testschritten "`TGR find first request to path {}`", "`TGR find next request to path {}`" und "`TGR find last request to path {}`" erledigen.

- Baue vor "I control the health endpoint" einen weiteren Schritt ein, der eine Abfrage gegen `/service/hello` schickt
  - *Mit zwei verschiedenen Anfragen können wir sicherstellen, dass wir die richtige Antwort kontrollieren.*
- Ändere das Suchen nach der Nachricht: `When TGR find first request to path "/service/.*"`
  - *Führe jetzt den Test einmal aus: Er wird fehlschlagen. Zuerst wird die hello Abfrage gefunden, die einen anderen Inhalt als der Health Check Call hat. Das gilt natürlich nicht wenn deine Abfrage nicht generisch war.*

- Nun führen wir zuerst den *Health Check Call* vor der hello Abfrage aus: Checke, ob das Attribute "$.body" gleich "OK" ist.
  - *Ausführen nicht vergessen. Der Test sollte jetzt erfolgreich sein.*

- Würden wir vorherigen Schritt nicht machen wollen, können wir mit: `When TGR find next request to path "/service/.*"` und/oder mit `When TGR find last request to path "/service/.*"` die korrekte Request ermitteln
  - *Dies sucht die Nachricht ausgehend von der aktuellen Position des Zeigers, nicht vom Anfang der Nachrichten.*

- den gesuchten Request finden und die Response vergleichen

Da der "current request/response" Zeiger auch über Scenarios hinweg agiert, ist es sinnvoll, zu Beginn eines Scenarios den Schritt "`TGR clear recorded messages`" auszuführen und die vorangegangenen Nachrichten zu löschen.

Zusatzaufgabe

- Schreibe ein neues Feature Login "/service/login" mit username und password als QueryParameter
- Teste den ResponseCode auf 400

Stage 4: Workflow UI/Proxy LogI/Rbel-Path

Jetzt wollen wir weitere REST Anfragen an die URL schicken und uns die Workflow UI anschauen. Die Workflow UI und auch die WebUI ist ein Frontend, welches gerade beim Aufsetzen der Tests sehr hilfreich sein kann.

Dafür setzen wir in der tiger.yaml folgenden Eintrag auf true:

```
lib:
  activateWorkflowUi: true
  trafficVisualization: true
```

Nach dem Starten des Scenarios oder des Featurefiles öffnet sich die Workflow UI im Defaultbrowser. Erklärungen zur Workflow UI gibt es im Abschnitt Workflow UI im Manual. Erklärungen zu, Proxy Log sind im Abschnitt Standalone Tiger Proxy Log zu finden.

RBel-Path ist eine an XPath/JsonPath angelehnte Abfragesprache. Mit ihr kann man in beliebige, mit dem Tiger-Proxy aufgefangene Nachrichten hineinschauen. Der entscheidende Unterschied ist, dass die Struktur unabhängig vom eigentlichen Aufbau her aufgedröselt wird. Das Root-Element ist in unserem Fällen hier immer direkt die HTTP-Nachricht. Mit "`$.body`" erhalten wir also den HTTP-body, mit "`$.header`" die Header-Struktur. Wenn die Nachricht eine HTML-Seite wäre, könnten wir mit "`$.body.html.header`" das Header-Tag der HTML-Struktur auswählen. Wir bauen hier ein einfaches Beispiel, mit dem wir in die JSON-Struktur der Rückgabe des Testtreibers hineinschauen. Mehr über Rbel-Path kannst du im Abschnitt Understanding RBelPath nachlesen.

- Neuen Testschritt hinzufügen: `TGR print current response as rbel-tree`
  - *Dieser Schritt gibt die aktuell ausgewählte Antwort in der Kommandozeile als Baumstruktur aus*

- Kurzes Schauen auf den Baum in der Konsole. Zum Vergleich kann man sich den RBel-Log daneben legen (zu finden unter target/rbellogs).

- Nun ersetzen oder ergänzen wir die Vergleiche aus der vorigen Stage. Zuerst den Login
  - `Then TGR current response with attribute "$.body.status" matches "400"`
  - Mehr Infos zur Traffic Analyse in der Proxy Log findet ihr in diesem Video.

Für die folgenden Tests bauen wir einen vollständigen Login- und Registrierungstest. Dafür können wir uns die Schnittstellen via OpenApi anschauen (über die Workflow UI starten, und dann "/swagger-ui/index.html" an die Demo-Url hängen).

Dann brauchen wir:

- Eine Vorbedingung "user x existiert nicht bzw. wird gelöscht" (brauchen wir falls wir "replay" in der WorkflowUi machen wollen)

- Einen parametrisierten Login-Schritt mit Username und Passwort (haben wir aus vorherigem Stage)
- Einen Login-Lauf mit einem nicht bekannten User, der fehlschlagen soll (haben wir aus vorherigem Stage)
- Einen Registrierungslauf, der erfolgreich sein soll, und ein anschließendes Login, das ebenfalls erfolgreich sein muss

Hinweis: Versuch so viel wie möglich die vorgefertigten GlueCode Steps des Tigers zu nutzen und so wenig wie möglich eigenen GlueCode zu schreiben.

Stage 5: Configuration & Placeholder

Per Platzhalter werden Daten, vllt username/pw, eingefügt. Und in der Rückgabe werden dann nicht feste Werte verglichen sondern einfach genau diese Platzhalter wieder. `mvn verify` nicht vergessen.

Eines der Kern Features, die Tiger mitbringt, ist das Einlesen und Verwalten von Daten & Konfiguration. Dieses Feature nutzen wir schon die ganze Zeit, nämlich um die tiger.yaml einzulesen und die Testumgebung zu manipulieren. Die `tiger.yaml` ist die einzige Datei, die automatisch eingelesen wird. Der Basekey der `tiger.yaml` ist "`tiger`". Diesen kann man benutzen, um im feature-File auf Variablen aus dem `tiger.yaml` zuzugreifen. Um Testdaten oder Default-Werte aus weiteren Dateien einzulesen, müssen wir dafür Sorge tragen, dass diese anderen Dateien auch eingelesen werden. Dazu tragen wir die entsprechenden Dateien unter `additionalConfigurationFiles` in der `tiger.yaml` ein. Das wird im Abschnitt Additional YAML files näher erklärt.

- Lege eine neue YAML-Datei an in welcher du einen Nutzernamen und ein Passwort hinterlegst. Die Struktur kannst du dir hierbei selbst überlegen.
- Trage die Datei in die `tiger.yaml` unter `additionalConfigurationFiles` ein:

```yaml
additionalConfigurationFiles:
  -
    filename: demoData.yaml
    baseKey: demo
```

    - Der Filename muss auf die einzulesende Datei verweisen, relativ vom Root-Verzeichnis der Testsuite aus.
    - BaseKey ist ein Schlüssel welcher als Prefix für alle Schlüssel in einer YAML-Datei dient. Der BaseKey für die `tiger.yaml` ist `tiger` .
    - Man kann auch auf einen BaseKey verzichten, in diesem Falle wäre der Basekey "`tiger`". Es ist lediglich ein Mechanismus, um die Verwaltung von Namespaces zu vereinfachen.
- Ersetze nun in dem feature-File die Username- und Password-Parameter beim Einloggen und Registrieren mit Platzhaltern der Form `${gewählterBaseKey.gewählterUser.username}` . Diese Platzhalter sollen auf die Werte aus der Daten-Yaml Datei verweisen.
- Im Gluecode müssen nun die `{string}` durch `{tigerResolvedString}` ersetzt werden, die Methodenparameter bleiben String. Durch `tigerResolvedString` erkennt Tiger, dass dieser String zuvor aufgelöst werden muss, so dass auch tatsächlich Werte für die entsprechenden Platzhalter hinterlegt werden.
- Fertig! Führe nun einen Testlauf aus und schaue in das RBel-Log hinein. Dort sollten die ersetzten Werte auftauchen (wenn du alles richtig gemacht hast).

Zusatzaufgabe

- Für weitere Tests kann es nützlich sein, mehrere Test-User zu hinterlegen.
    - Baue in die Data-YAML eine Map an Test-Usern ein
    - In dem Aufruf des Glue-Codes musst du nun den Namen angeben. Verwende hierfür ebenfalls eine Variable!
- Man kann natürlich auch eine Variable verwenden, um diese dann im TigerConfiguration Editor zu ändern und mittels Replay in der WorkflowUI ein Scenario erneut abzuspielen. Probiere es aus. Das funktioniert auch mit Variablenänderungen in der Feature-Datei selbst. Wichtig ist hier: Es sollte keine Zeilen gelöscht/hinzugefügt werden, sonst ist der korrekt Test nicht mehr gewährleistet, weil Cucumber erkennt und identifiziert Tests teilweise anhand des Speicherorts der Feature-Datei und der Zeilennummer, in der sich die Tests befinden.

Mehr Infos zur TigerConfiguration findet ihr in diesem Video.

Stage 6: Umgebungsabhängige Konfiguration

Wir führen zwei Testumgebungen ein: "local" und "external". Es sollen unterschiedliche Daten in Abhängigkeit der Umgebung gewählt werden. Weiterhin muss natürlich die Umgebung selbst angepasst werden.

Es gibt in unserem Projekt jetzt zwei verschiedene Arten von Testumgebungen: Eine externe und eine lokale Testumgebung. Das ist eine sehr normale Situation: Lokal will man beim Entwickeln der Testsuite mit Simulatoren arbeiten, auf dem Buildserver oder bei einem echten Abnahmetest mit einem externen System. Mit dem oben dargestellten Konfigurations-Mechanismus hat man eigentlich schon alle Tools an der Hand, um ziemlich elegant mit diesem Problem umzugehen. Wir wollen das hier für uns einmal nachbauen.

- Lege zwei weitere YAML-Dateien an: `tiger-local.yaml` und `tiger-external.yaml`
- Kopiere aus der bisherigen tiger.yaml den gesamten `servers` -Node in die `tiger-local.yaml` . Und lösche die Server aus der tiger.yaml. Diese YAML nutzen wir, um lokal eine aktiv verwaltete Umgebung anzustarten.
- In die `tiger-external.yaml` kopierst du auch den `servers` -Node auf

```yaml
servers:
  demo:
    type: externalUrl
    source:
      - http://localhost:8080
    healthcheckUrl: http://localhost:8080/service/status
    healthcheckReturnCode: 200
```

    - Führe das demo.jar aus. Der Defaultport ist 8080.

- Nun müssen wir noch die richtige der beiden YAML-Dateien einlesen. Wir fügen dazu in der `tiger.yaml` einen neuen Eintrag unter `additionalConfigurationFiles` ein. Das besondere: Der Dateiname wird selber wieder mit einem Platzhalter ergänzt: `tiger-${environment}.yaml mit baseKey "tiger".`
- Jetzt kannst du das Setup schon ausprobieren: `mvn verify -Denvironment=local` und im IntelliJ mit `environment=local` in den Environment variables in den run configurations.

Das ist schon sehr cool, aber es gibt eine entscheidende Schwäche: Was ist mit Default-Werten? Vielleicht will ich immer local nehmen, außer ich will tatsächlich gegen die externe -Umgebung gehen.

- Lege eine neue Datei `defaults.yaml` an. Diese binden wir auch wieder als additionalConfigurationFiles mit ein, aber diesmal OHNE baseKey, dafür an oberster Stelle der additionalConfigurationFiles.
- In diese Datei schreibst du den default-Wert für `environment` mit rein.
- Und ausprobieren: `mvn verify` startet nun die Testsuite mit der Default-Umgebung.

Stage 7: RbelPath reloaded

Wir asserten direkt in das zurückgegebene JWT hinein.

Oben wurde schon erwähnt, dass RBel-Path gerade bei ineinander verschachtelten Datenstrukturen seine Fähigkeiten voll ausspielen kann. Das werden wir jetzt bei dem zurückgegebenen JWT (Json Web Token) demonstrieren. Ein JWT ist eine kompakte Struktur, mit der signierte Daten ausgetauscht werden können. Sie besteht aus drei Teilen, Base64-URL kodiert und mit einem Punkt unterteilt. Die Teile heißen Header, Body und Signatur. Der Header beschreibt, wie die Signatur zu checken ist, der Body ist die signierte Payload, die Signatur beglaubigt den Body (oder auch Body und Header).

- Schaue zuerst auf das Rbel-Log und betrachte das JWT (sollte als Antwort auf /login kommen). Das JWT kann auf jwt.io näher untersucht werden.
- Rbel-Tree zeigen (`TGR print current response as rbel-tree`)
- Username im Token selbst asserten



Zusatzaufgabe

- Asserte auf einen korrekten Algorithmus im Header
- Recursive-descent: Suche nach jedem Vorkommen von `name` in der gesamten Nachricht (`$..nbf` gibt alle Einträge von nbf in der gesamten Struktur zurück)

Mehr Infos zum RbelPath und JEXL findet ihr in diesem Video.

Stage 8: JSON-Checker (optional)

Noch eine Stufe eleganter: Wir vergleichen gleich das ganze JSON, sprinkeln ein paar Platzhalter rein. Dabei müssen wir natürlich noch einen Zeitstempel ignorieren.

Hierfür gehen wir zu dem erfolglosen service/login und schauen uns die Response an.

Als Hilfsmittel beim Testen von Protokollen wurde ein direkter JSON-Vergleich aus dem feature-File heraus entwickelt. Das hilft dabei, sehr ausdrucksstark eine Schnittstelle zu beschreiben.

- Neuen Vergleich einbauen "`TGR current response at "$.body" matches as JSON:`"
    - Beachte dabei, dass Strings die über mehrere Zeilen gehen (im feature-File), mit """ beginnen und enden
- Für das Datum entweder ein "cooles" Regex bauen oder per "`${json-unit.ignore}`" den Wert ignorieren.

Tipps

Wollt ihr in eurem Gluecode direkt auf den TigerGlueCode zugreifen/nutzen, macht das am besten mit der @Steps Annotation von serenity. Diese instanziiert die annotierten Klassen.

```
import net.serenitybdd.annotations.Steps;

public class TestsuiteGlueCode {

    @Steps
    HttpGlueCode httpGlueCode;
    @Steps
    RBelValidatorGlue rBelValidatorGlue;
...
}
```

Es gibt für die Bearbeitung der tiger.yaml ein JSON-Schema das mit eingebunden werden kann: https://json.schemastore.org/gematik-tiger.json